Preliminary draft 22:34 9 April 2026

9 April 2026
author.email@cern.ch

DFX4ML-ARCH

Omitted for blind review
CERN, CH-1211 Geneva, Switzerland

Keywords: FPGA, Partial Reconfiguration, Dynamic Function Exchange, PYNQ, Xilinx

Summary

DFX4ML-ARCH presents an FPGA architecture designed to enable partial reconfiguration for machine learning workloads. The project incorporates a self-reconfiguration mechanism, allowing the FPGA fabric to partially reprogram itself. It targets AMD FPGAs equipped with the ICAP3 interface and leverages the PYNQ framework for system integration.

Disclaimer: The technical content, architecture designs, implementation decisions, and all intellectual contributions in this report are entirely the work of the authors. Generative AI tools were used solely for grammar correction and readability enhancement of the written text. No AI-generated technical content, code, or design decisions are included.

Template: This document uses the CERN ATS Note LaTeX template, available at https://www.overleaf.com/latex/templates/cern-ats-note/yzjmgkrvtsqg.

Contents

1 Project Overview **3**
1.1 Project Structure . . . 3
1.2 Naming Convention . . . 4

2 Hardware Architecture **4**
2.1 DFX Manager . . . 5
2.1.1 DFX Manager Core . . . 6
2.2 DFX Streamer(s) . . . 13
2.3 DFX_CTRL . . . 13
2.4 Other Relevant IPs . . . 14

3 Driver Architecture **15**

4 Build Procedure **17**
4.1 Quick Start 17
4.2 ML Framework Integration 19
4.2.1 User Build Tcl 19
4.3 Board-Specific Script 21

1 Project Overview

DFX4ML is an FPGA architecture aiming to execute machine learning workloads on the FPGA by exploiting partial reconfiguration to fit the FPGA's resources to large machine learning models. The model will be divided into multiple pieces and then sequentially set up on the reconfigurable region to execute the machine learning workload.

In current version, the hardware and driver are built and tested under following specification:

1. FPGA model is tested on Zynq UltraScale+ (kria260)
2. Hardware model is composed in Vivado 2023.2
3. The target FPGA is running Ubuntu 22.04 with the PYNQ framework

This project is composed of sections:

1. **Hardware Architecture**: it describes the architecture idea, hardware implementation details, and reconfigurable parameters.
2. **Software Driver**: This driver mainly controls our hardware IP and Xilinx IP in the programmable logic. In this version, we provide only the PYNQ driver.
3. **Build Procedure**: This section provides our formal approach to enable DFX4ML-ARCH to integrate with existing Hardware Machine Learning Frameworks and to specific FPGA board.

1.1 Project Structure

Project Structure

```
.
+-- hw                                # for hardware resource
|   +-- bd_src                         # Vivado block design script
|   |   +-- dfx4ml                      # static and pr connection script
|   |   +-- dfx_region                  # par block design script
|   |   +-- dfx_unified                 # static region block design script
|   +-- build_script                   # block design for specific board script
|   |   +-- kv260
|   +-- ip_src                         # verilog ip script
|       +-- dfx_icap
|       +-- dfx_mng
|       +-- dfx_streamer
|       +-- dfx_streamer_mreset
|       +-- dfx_streamer_mshut
|       +-- dfx_streamer_s2m
|       +-- dfx_streamer_s2m_free
|       +-- dfx_streamer_sreset
|       +-- dfx_streamer_sshut
|       +-- fourSp
```

```
+-- lib                                # python library for hw and sw generation
+-- sw                                 # for software resource
    +-- driver
```

1.2 Naming Convention

DFX4ML-ARCH adopts the following naming conventions across its Python and Verilog codebase.

- **Python classes and Verilog module names** use *Pascal_Snake_Case* (e.g., `Hw_Build_Helper`, `Dfx_Streamer`).
- **Python variables and methods** use *snake_case* (e.g., `build_folder_path`, `run_build()`).

Note that this convention is not strictly enforced throughout the entire codebase. Some legacy modules were written before this rule was established and may not fully conform. A refactoring pass to align these modules is planned for a future minor version.

2 Hardware Architecture

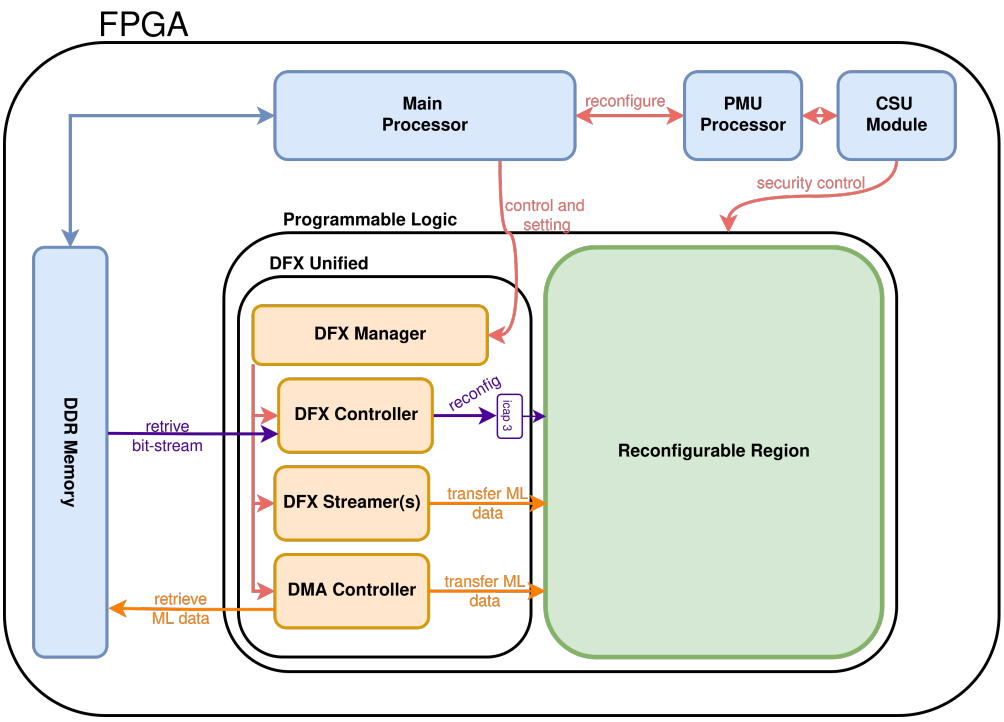

Figure 1: DFX4ML-ARCH overall architecture

Fig.1 illustrates the overall hardware architecture of DFX4ML-ARCH. The hardware architecture is composed of 4 major parts:

1. **Main Processor**. This project focuses on the Xilinx SoC with the PYNQ framework. The PYNQ framework allows users to manage memory allocation and initialization. Moreover, the processor is responsible for controlling Programmable Logic and communicating with the PMU processor.

2. **PMU and CSU**. The Platform Management Unit (PMU) and Control Security Unit (CSU) are the Xilinx SoC's management systems. The PMU is a dedicated processor used to manage the system's routine tasks and secure the system at the hardware level. In this project, we have to command the PMU to orchestrate the Programmable Logic permissions.

3. **Programmable Logic**. To support partial reconfiguration, the DFX4ML-ARCH is divided into 2 region:

   (a) **Static Region**. The region that is always active and contains the control logic and temporary memory storage. We propose four main components:

      i. **DFX Manager**: it manages all PL components: (1) commands the DFX Controller, (2) controls the DFX Streamer, (3) controls the DMA Controller, (4) profiles the system, (5) interrupts the processor, and (6) receives commands and metadata from the processor.

      ii. **DFX Controller**: This Xilinx IP receives commands from the DFX Manager. When it is triggered, the system loads the partial bitstream file from main memory (DDR RAM) and reprograms the partial region through the ICAP3 port.

      iii. **DFX Streamer(s)**: it is the storage based on BRAM (on-chip memory) used to hold data temporarily while the partial region is reconfiguring.

      iv. **DMA Controller**: this Xilinx IP is used to transfer data from DDR RAM directly to the partial region.

   (b) **Reconfigurable Region (RP)**. This region is dedicated to machine learning workload. The inputoutput only comes from the AXI-Stream.

2.1 DFX Manager

DFX Manager is the IP used to orchestrate the entire Programmable Logic function. Fig. 2 illustrates the overall components in the manager.

The DFX Manager is composed of 5 major components:

1. PS Connector: It communicates with the main processor (PS) through the AXI-LITE slave protocol. The code is at `hw/ip_src/dfx_mng/s_axi_{read,write}.v`

2. DFX Streamer Connector: The connector directly passes signals from the DFX Manager Core through control signals to trigger load/store operations, and also receives the data-holding status from the streamers.

3. DMA Connector: The DFX Manager Core requires multiple steps to set up the DMA controller properly; therefore, the DMA connector is responsible for managing the

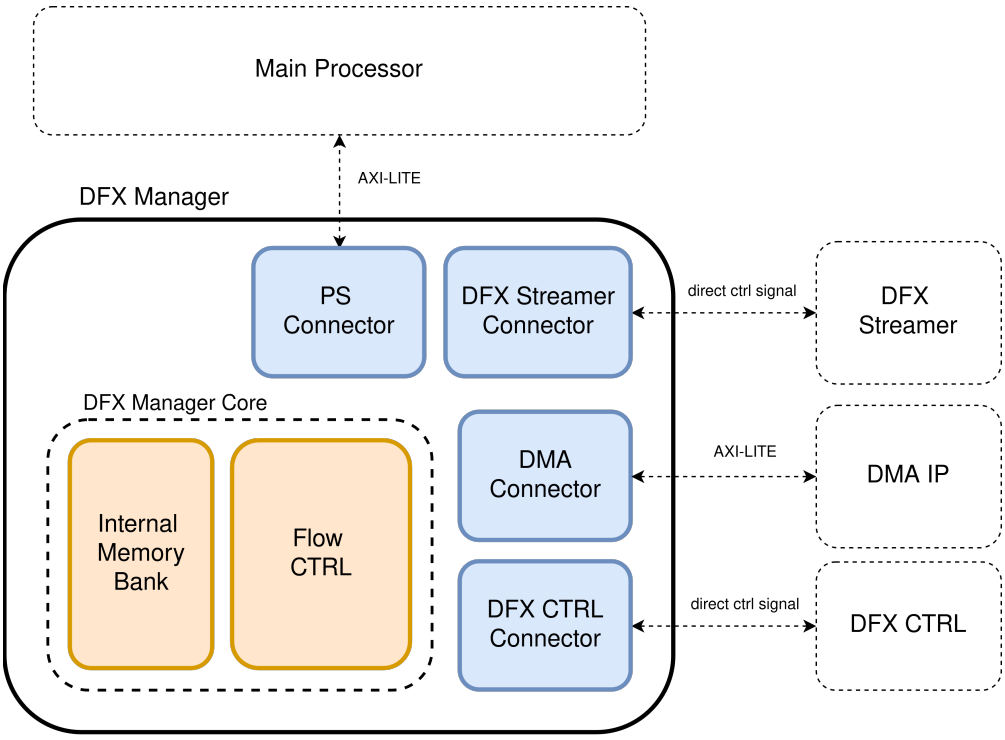

Figure 2: DFX Manager

sequence of AXI-LITE packets sent to the DMA IP. The code is at `hw/ip_src/dfx_mng/m_axi_{read,write}.v`

4. DFX CTRL Connector: The connector directly passes signals from the DFX Manager Core through control signals to trigger the DFX CTRL IP to decouple and reconfigure the partial region. This connector also receives data back from the target IP to indicate that the reconfiguration is finished.

5. DFX Manager Core: it contains the internal register memory used to control the flow of the system and provide data for the slave IPs. Moreover, Flow CTRL is responsible for orchestrating the slave IPs to work in the correct order.

2.1.1 DFX Manager Core

Register Architecture (internal register bank)

We propose the register architecture that resides in the internal memory bank. Users can access these via memory-mapped I/O. The TCL build script sets the default address at $0xA000_0000$ with a 64 KB address range.

With a 64 KB range, the address space is categorized into 3 main sections:

1. **bankId** (bits 14–15): the system supports up to 4 banks. Each bank is a group of rows that belong to the same functional group.

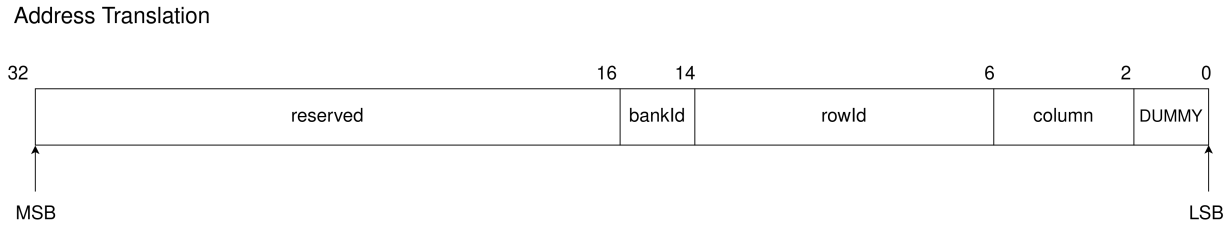

Figure 3: DFX Manager Core's internal register address translation

2. **rowId** (bits 13–6): each bank supports up to 256 rows. A row is a group of registers; however, different rows in the same bank do not have to follow the same schema.

3. **column** (bits 5–2): each row supports up to 16 registers. Each register in the row does not have to be the same size. However, the register size must not exceed 4 bytes (32 bits).

Fig. 4 illustrates the DFX Manager's internal memory. The system is composed of two memory banks: Bank 0 is dedicated to IP control, the debugging system, overall running configuration, interrupt settings, and flow control memory; Bank 1 is composed of several rows sharing the same register pattern, used to store data and metadata for reconfigurable module initialization and slave static IP commands.

Bank 0 — Global Control Registers

This bank is mainly used to control the flow control system and slave IP identifiers. We provide 9 main dedicated registers, which are accessible from the processor (PS) via the AXI-LITE protocol. In this current version, the `DFX_CTRL_ADDRESS` and `ROUND_TRIP` registers are deprecated. The code is at `hw/ip_src/dfx_mng/dfx_mng_core.v`

Base offset: `0x0000`

Table 1: Bank 0 — Global Control Registers

Address	Register	Width	Access	Description
0x0000	`CTRL_REG`	4 bit	**WO**	Control register. **0** = CLEAR **1** = SHUTDOWN **2** = START UP

Table 1: Bank 0 — Global Control Registers (continued)

Address	Register	Width	Access	Description
0x0004	STATUS_REG	4 bit	**RO**	System status flags/ flow state machine: STATUS_SHUTDOWN STATUS_REPROC STATUS_W4SLAVERESET STATUS_W4SLAVEOP STATUS_CLEAR_MGS STATUS_INITIALIZE_MGS STATUS_INITIALIZE_DMA STATUS_SET_DMA_LOAD STATUS_SET_DMA_STORE STATUS_TRIGGERING STATUS_WAIT4FIN
0x0008	MAIN_COUNTER	$\lceil \log_2(sw) \rceil$ bit	**RO**	Main loop counter. Width is $\lceil \log_2(sw) \rceil$ bits.
0x000C	END_COUNTER	$\lceil \log_2(sw) \rceil$ bit	**R/W**	End-of-loop counter. Defines the terminal count value for the main counter.
0x0010	DMA_ADDRESS	32 bit	**R/W**	Base DMA address register (32-bit).
0x0014	DFX_CTRL _ADDRESS	32 bit	**R/W**	DFX control base address (32-bit).
0x0018	INTR_ENA	1 bit	**R/W**	Interrupt enable. Set to **1** to enable notification on finish execution.
0x001C	INTR	1 bit	**RO**	Interrupt status. Indicates system has finished in the current round (read/WOC). Set to **1** when finished.
0x0020	ROUND_TRIP	16 bit	**R/W**	Round-trip invocation counter. Counts the number of times the IP has been invoked.

Bank 1 — Reconfigurable Module Slot Registers

This bank is dedicated to reconfigurable module data and metadata; therefore, it is composed of multiple register rows with the same pattern. Each row is supposed to contain the necessary data and metadata for a single reconfigurable module (RM). The row index is also considered the workload execution sequence. In this current version, the register SLOT_STATUS is deprecated. The code is at `hw/ip_src/dfx_mng/{slot,slotTable}.v`

Base offset: `0x4000 + 0x40*rm_i`

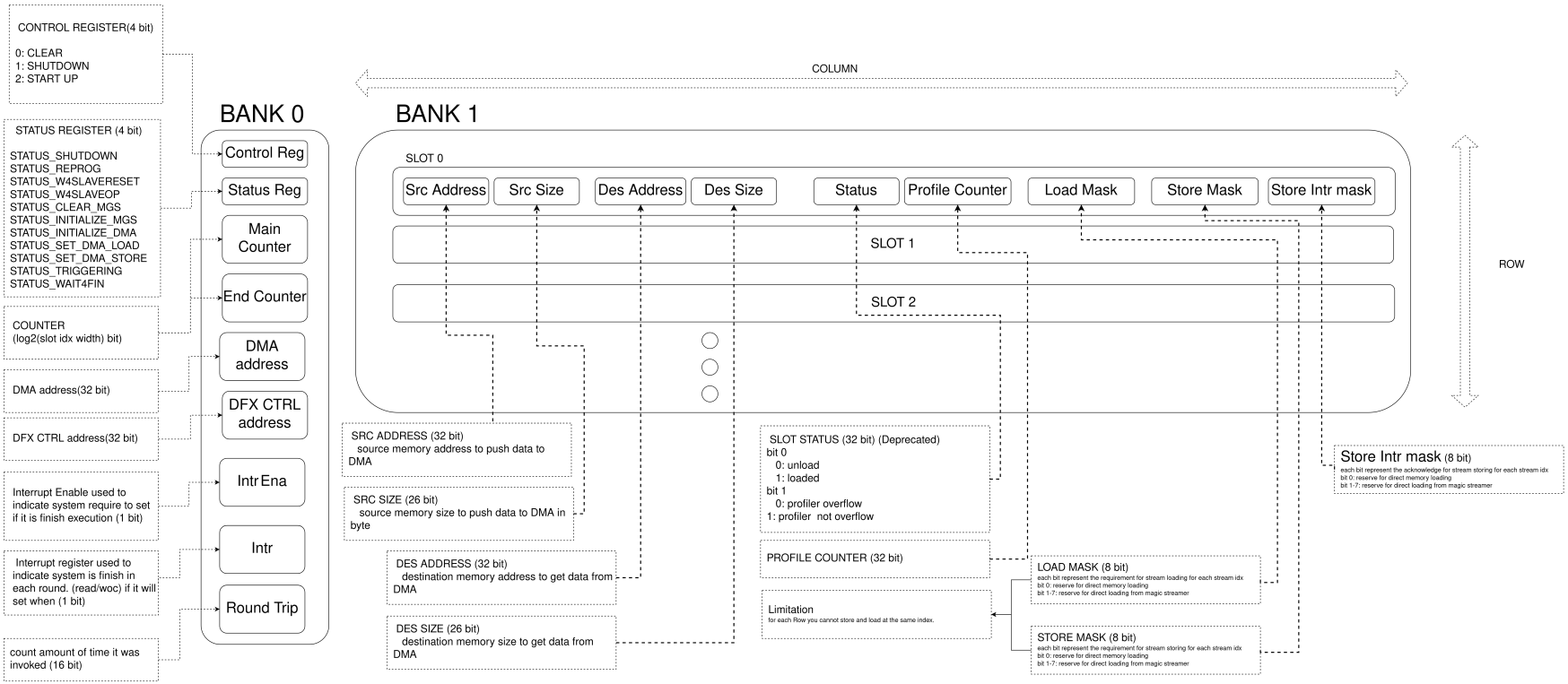

Figure 4: DFX Manager Core's internal register overview

Slot Offset	Register	Width	Access	Description
DMA Transfer — Source				
+0x00	SRC_ADDRESS	32 bit	R/W	Source memory address. Address in system memory from which data will be pushed to the DMA engine.
+0x04	SRC_SIZE	26 bit	R/W	Source transfer size in bytes. Specifies the number of bytes to push from source memory to DMA.
DMA Transfer — Destination				
+0x08	DES_ADDRESS	32 bit	R/W	Destination memory address. Address in system memory where the DMA engine writes the output data.
+0x0C	DES_SIZE	26 bit	R/W	Destination transfer size in bytes. Specifies the number of bytes the DMA engine writes to destination memory.
Slot Status & Profiling				
+0x10	SLOT_STATUS	32 bit	R/W	Bit 0: load state (0 = unload, 1 = loaded). Bit 1: profiler overflow flag (0 = overflow, 1 = no overflow).
+0x14	PROFILE_COUNTER	32 bit	R/W	Profiling cycle counter. Counts partial region reconfigure's usage cycles for this RM.
Stream Masks				
+0x18	LOAD_MASK	8 bit	R/W	Each bit at index i in this register indicates the data demand from streamer i. The first bit (LSB) is reserved for DMA. 1 means required; 0 means otherwise.
+0x1C	STORE_MASK	8 bit	R/W	Each bit at index i in this register indicates a request to store data to streamer i. The first bit (LSB) is reserved for DMA. 1 means required; 0 means otherwise.
+0x20	STORE_INTR_MASK	8 bit	RO	Holds the completion status of the DFX Streamer and DMA IP following the same bit pattern as STORE_MASK. This register is used to indicate whether this RM has finished.

Access Permission Legend

R/W	Read / Write	Register can be both read and written by the host.
RO	Read Only	Register is read-only; writes are ignored.
WO	Write Only	Register is write-only; reads return undefined values.

Flow Control (state machine)

Flow Control orchestrates the slave IP steps, such as DFX Controller, DMA IP, and DFX Streamer (formerly Magic Streamer), to execute initialization and operation commands sequentially at the cycle-accurate level. The code is at `hw/ip_src/dfx_mng/dfx_mng_core.v`.

Fig. 5 illustrates the high-level state machine inside the DFX Manager Core, and we can categorize the system commands into 3 main parts:

1. SHUTDOWN state: The DFX Manager will not issue any commands and will not follow any commands from an external source, except commands from the processor. All internal registers can be read or written following the rules in the previous section (Access Permission Legend). Except for `CTRL_REG`, if there are register accesses when the system is not in SHUTDOWN state, the system will return unpredictable values and our driver will discard them. We apply this restriction because we aim to optimize hardware resource usage.

2. DFX CTRL states: these states are depicted in the DFX CTRL handshake section (green block) and are dedicated to sending commands to trigger the DFX CTRL IP. The commands aim to reconfigure the partial region and track/synchronize the reconfiguration status.

3. MGS and DMA states: these states are depicted in the MGS & DMA initialization (brown block) and are dedicated to sending commands to initialize and trigger the DMA IP and DFX Streamer (formerly Magic Streamer (MGS)). The commands aim to prepare or clear the internal metadata memory of the DFX Streamer, set up the DMA IP metadata obtained from internal register bank 1, and wait for the reconfigurable module to finish.

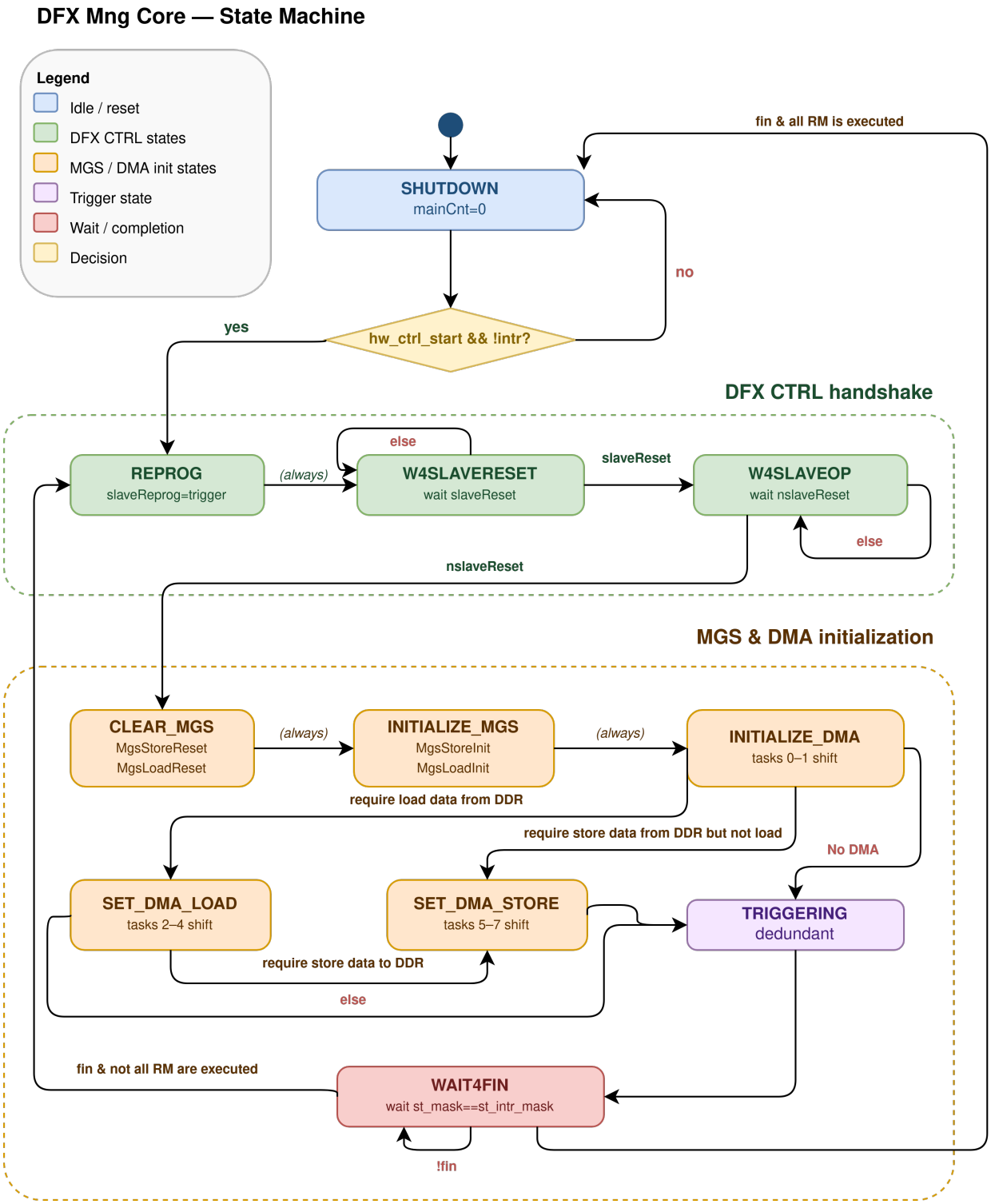

Figure 5: DFX Manager flow control (state diagram)

2.2 DFX Streamer(s)

DFX Streamer(s) is the temporary storage residing in the Programmable Logic static region and was designed to utilize BRAM resources for storage and a small number of flip-flops to hold its transfer metadata. The code is at `hw/ip_src/dfx_streamer/dfx_streamer.v`

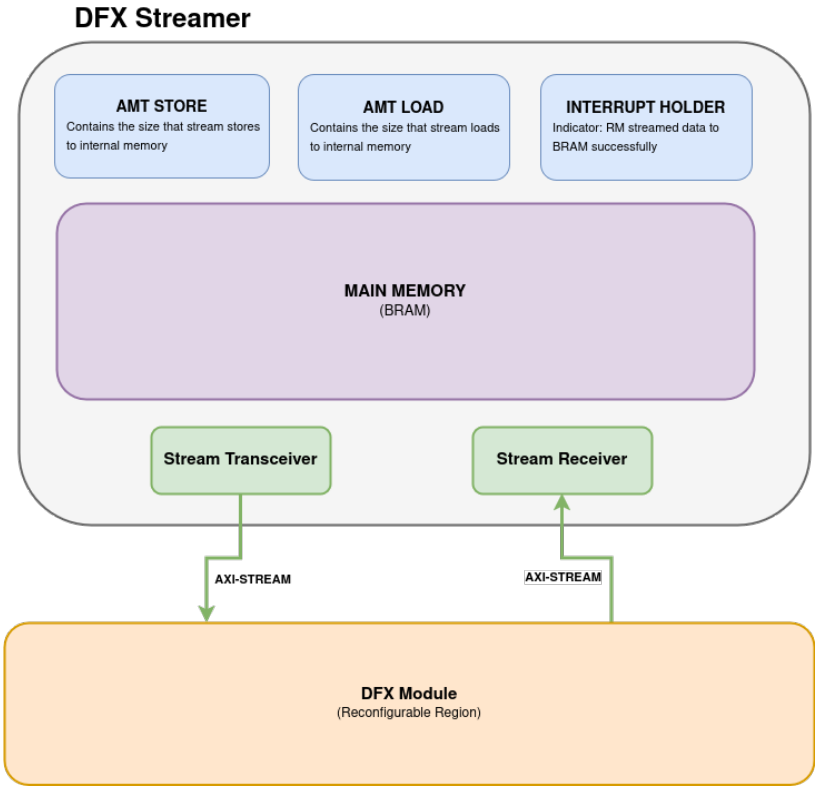

Figure 6: DFX Streamer Architecture

Fig. 7 illustrates the overall architecture of the DFX Streamer. The **meta-data blocks** (blue boxes) reside in the static region, using flip-flops to track the amount of load/store operations and RM status (INTERRUPT HOLDER). Meanwhile, the **MAIN MEMORY** (purple block) also resides in the static region, but uses BRAM to store the temporary data.

In contrast, the Stream Transceiver and Stream Receiver connection ports are open to the partial region via the **AXI-STREAM** interface. In the current version, the AXI-STREAM **tkeep** and **tlast** signals are **mandatory**. If they are not set up, the system will get stuck.

2.3 DFX_CTRL

DFX_CTRL is the Xilinx DFX IP used to retrieve the bitstream file from DDR memory and reconfigure the partial reconfigurable region using the ICAP3 port. To achieve this, we need to configure the IP's internal memory to match our requirements.

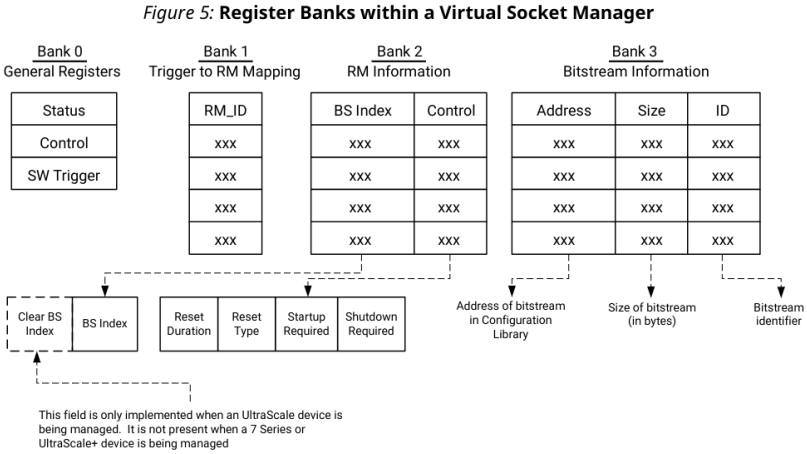

Figure 7: DFX Controller Architecture (copied from [1])

Fig. 7 [1] illustrates the internal memory of the DFX CTRL IP. It is composed of 4 memory banks:

1. Bank 0 contains general registers enabling software to control the IP and check its status.
2. Bank 1 is the RM Mapping; each slot in this bank is a pointer to a slot in Bank 2.
3. Bank 2 contains RM information. The control field is the setting of the start-up/shutdown procedure, and our driver sets these criteria as follows:
   (a) Reset Duration: 1 cycle
   (b) Reset Type: active low
   (c) Startup Required: No
   (d) Shutdown Required: No

   The BS Index field contains the index of the target slot in Bank 3.
4. Bank 3 contains the bitstream information:
   (a) Address: the address of the bitstream file in DDR memory
   (b) Size: the size of the bitstream file in bytes
   (c) ID: bitstream identifier

2.4 Other Relevant IPs

In DFX4ML, there are 3 other IPs that support system creation:

1. icapWrap is the module that serves as an interface for the DFX Controller to self-reconfigure. `hw/ip_src/dfx_icap/dfx_streamer.v`

2. Stream_Master_Dummy is the module used as a dummy plug on the partial region AXI-Stream master side. It is built to prevent unpredictable signals when the target reconfigurable module does not require certain interface ports.
   `hw/ip_src/dfx_streamer_mshut/streamer_mshut.v`

3. Stream_Slave_Dummy is similar to Stream_Master_Dummy but for the AXI-Stream slave side.
   `hw/ip_src/dfx_streamer_sshut/streamer_sshut.v`

3 Driver Architecture

Memory Address

Our build script sets a default base address of `0xA000_0000` for the DFX Unified IP, which includes DFX Mng IP, DFX Ctrl IP, DMA IP, PR Reset, and PR DECUP.

Table 3: DFX Unified IP Address Map (Base: `0xA000_0000`)

IP	Offset	Absolute Address	Address Range
DFX Ctrl	`0x0_0000`	`0xA000_0000`	`0xA000_0000` – `0xA000_FFFF`
PR DECUP	`0x1_0000`	`0xA001_0000`	`0xA001_0000` – `0xA001_FFFF`
PR Reset	`0x2_0000`	`0xA002_0000`	`0xA002_0000` – `0xA002_FFFF`
DMA	`0x3_0000`	`0xA003_0000`	`0xA003_0000` – `0xA003_FFFF`
DFX Mng	`0x4_0000`	`0xA004_0000`	`0xA004_0000` – `0xA004_FFFF`

Driver Component

This section illustrates the PYNQ driver overview used to command the DFX4ML-ARCH hardware.

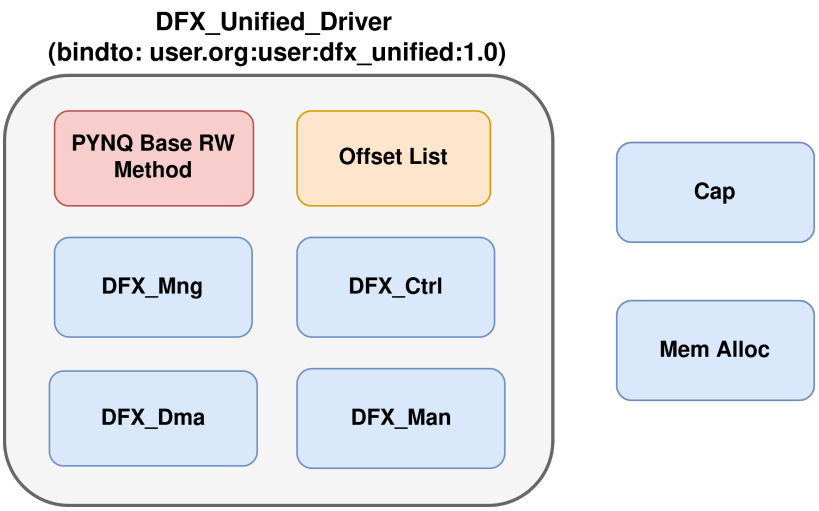

Figure 8: DFX4ML-ARCH Driver Overview

DFX_Unified_Driver: This class controls the DFX Unified IP's operations. As described in the Hardware Architecture section, the DFX Unified IP is composed of 4 sub-IPs: DFX Manager, DFX Controller, DFX DMA, and DFX Streamer(s).

Therefore, **The Unified Driver** is composed of 4 sub-drivers (blue boxes): DFX Manager Driver to control the Reconfigurable Module's flow; DFX Controller Driver to manage the partial bitstream; DMA Driver to manage the memory data transfer, used for debugging purposes only; and DFX Manual, used to manually decouple and reset the partial region, also for debugging purposes only.

the code is at
`sw/driver/dfx_unified.py`
`sw/driver/dfx_mng.py`
`sw/driver/dfx_ctrl.py`
`sw/driver/dfx_dma.py`
`sw/driver/dfx_man.py`

Cap is the helper function enabling designers to communicate with the PMU processor and reconfigure the `pcap_ctrl` register. After changing this register to 1, the FPGA will enable partial reconfiguration via the ICAP3 port. In contrast, if the register is set to 0, the FPGA will allow reconfiguration only from the processor (PS).

Mem Alloc is a helper function that facilitates users in allocating and copying data.

4 Build Procedure

4.1 Quick Start

We provide a Python interface that allows users to generate full and partial bitstream files (*.bin*), hardware handoff files (*.hwh*), DFX Ctrl configuration files, and PYNQ driver files.

Listing 1: Python Build Interface

```
1  from lib.hw_build import HwBuildHelper
2  from lib.sw_build import SwBuildHelper
3
4  # Hardware Build
5  hw_builder = HwBuildHelper(
6      build_folder_path        = "./build_prj",
7      dfx_root_path            = ".",
8      board                    = "kv260",
9      user_repo_path           = "",
10     req_gen_ip               = 1,
11     num_core                 = 4,
12     clk_frq                  = 99999001, # Hz
13     rm_index_width           = 2,
14     num_dfx_streamer         = 2,
15     interface_widths         = [32, 32],
16     applied_interface_widths= [32, 32],
17     storage_index_widths     = [10, 10],
18     num_actual_rm            = 2,
19     input_map_list           = [[0, -1], [-1, 0]],
20     output_map_list          = [[-1, 0], [0, -1]],
21     ip_map_list              = ["", ""],
22     test_mode                = 1,
23     vivado_path              = "<vivado path>",
24     export_folder_path       = "./export"
25 )
26 hw_builder.run_build()
27 hw_builder.package_export_files()
28 # Software Build
29 sw_builder = SwBuildHelper(export_folder_path="./export")
30 sw_builder.package_export_file()
```

The following example (Listing 1) demonstrates how to start building the DFX4ML-ARCH system. The file is at `quick_start.ipynb`. Users can use this file as a starting point. Lines 1–2 invoke the Hardware Build Helper and Software Build Helper respectively; Lines 5–27 allow users to build the hardware-related outputs (.bin, .hwh, DFX Ctrl configuration); and Lines 29–30 export the PYNQ driver files.

Table 4 describes each parameter of `HwBuildHelper`.

Table 4: HwBuildHelper Parameters

Parameter	**Description**
`build_folder_path`	Path to the directory where users want the Vivado project and other temporary working files. The folder will be created if the path does not exist.
`dfx_root_path`	Root path of the DFX4ML-ARCH source repository, used to locate IP cores, constraint files, etc.
`board`	Target FPGA board identifier (e.g., `kv260`).
`user_repo_path`	Path to the user-supplied IP repository, which typically contains the partial region's kernel. This path will be referenced only when parameter $test_mode = 0$.
`req_gen_ip`	Flag (`1`/`0`) indicating whether Verilog IP generation is required. The IPs are: ICAP interface, DFX Manager, DFX Streamer, and DFX Streamer Shut Plug. If you are running for the first time or have deleted/modified $build_folder_path$, YOU MUST SET THIS FLAG TO 1; otherwise this flag is optional.
`num_core`	Number of parallel synthesis/implementation jobs to run in Vivado.
`clk_frq`	Requested clock frequency in Hz for the FPGA (e.g., `99999001` $\approx$ 100 MHz).
`rm_index_width`	Bit-width of the reconfigurable module (RM) index, used to determine the number of rows in DFX Mng Bank 1 and DFX Ctrl slots. (e.g., `rm_index_width=3 -> 2 « 3 = 8 slots allocated`. However, the actual number of reconfigurable modules is not equal to this value; it is equal to num_actual_rm.)
`num_dfx_streamer`	Number of DFX Streamer instances instantiated in the static region.
`interface_widths`	List of data-bus widths (bits) for each DFX Streamer interface. Values must be powers of 2, starting from 8.
`applied_interface_widths`	List of effective data widths actually driven on each streamer interface; may differ from `interface_widths` when partial-width transfers require lesser.
`storage_index_widths`	List of address index widths (bits) for the internal buffer of each DFX Streamer, determining its storage depth (2^w entries).
`num_actual_rm`	Number of reconfigurable modules that will be loaded at runtime.

Parameter	Description
`input_map_list`	Per-kernel input routing table. Outer index = kernel ID; each entry is a list indexed by streamer ID, where the value is the kernel input port ID connected to that streamer (`-1` means the streamer is not connected to this kernel).
`output_map_list`	Per-kernel output routing table. Outer index = kernel ID; each entry is a list indexed by streamer ID, where the value is the kernel output port ID connected to that streamer (`-1` means the streamer is not connected to this kernel).
`ip_map_list`	Per-kernel IP core identifier strings. An empty string indicates that no additional IP is mapped to that kernel slot.
`test_mode`	Flag (`1`/`0`) enabling test mode, which inserts loopback logic for hardware verification without a loaded RM. The actual user kernel will be used only when $test_mode = 0$.
`vivado_path`	Absolute path to the Vivado installation directory (e.g., `"/tools/Xilinx/Vivado/2023.2/bin/vivado"`).
`export_folder_path`	Destination directory for the packaged output files (`.bin`, `.hwh`, configuration files, and PYNQ drivers). The folder will be created if the path does not exist.

4.2 ML Framework Integration

DFX4ML-ARCH provides the official solution for enabling Hardware Machine Learning frameworks to integrate their own sub-kernels into each Reconfigurable Module (RM).

To integrate the module, users must complete the following steps:

1. Provide the path to the user IP repository via `user_repo_path`. The folder at that path must contain the Vivado-exported IP folder, which in turn must include `src` and `xgui` sub-folders.

2. Provide the path to the TCL file that instructs the build script how to build the Reconfigurable Module via `user_build_tcl` (Listing 1).

3. Set `test_mode` to `0` (Listing 1) to enable the actual kernel instead of the loopback test logic.

4. Provide the IP name list in *ip_map_list* (Listing 1). The index corresponds to the Reconfigurable Module and the value is the IP name.

4.2.1 User Build Tcl

The user build TCL file must define the `create_dfx_region_bd` procedure with the following signature:

Listing 2: User Build TCL Procedure Signature

```
proc create_dfx_region_bd { \
    parentCell \
    block_name \
    clk_frq \
    num_dfx_streamer \
    interface_widths \
    input_maps \
    output_maps \
    ip_name \
} {
    # User implementation here
}
```

Table 5 describes each argument of the procedure.

Table 5: create_dfx_region_bd Arguments

Argument	**Description**
`parentCell`	This field is deprecated.
`block_name`	Name of the hierarchical block representing the Reconfigurable Module in the block design.
`clk_frq`	Clock frequency in Hz passed from `HwBuildHelper`, used to configure IP timing constraints inside the region.
`num_dfx_streamer`	Number of DFX Streamer interfaces available, matching the value set in `HwBuildHelper`.
`interface_widths`	List of data-bus widths (bits) for each streamer port, matching the value set in `HwBuildHelper`.
`input_maps`	Input routing map for this kernel slot (one row from `input_map_list`), indicating which streamer IDs feed which kernel input ports. The value corresponds to a first-level member of `HwBuildHelper`.
`output_maps`	Output routing map for this kernel slot (one row from `output_map_list`), indicating which streamer IDs receive which kernel output ports. The value corresponds to a first-level member of `HwBuildHelper`.
`ip_name`	IP core name string for this kernel slot, matching the corresponding entry in `ip_map_list`. The value corresponds to a first-level member of `HwBuildHelper`.

The fields *input_maps*, *output_maps*, and *ip_name* will be retrieved from the same index of the master list provided in (Listing 1).

Users must provide the RM implementation, which includes:

1. Create Block Design
2. Add ML IP module
3. Connect the I/O (clk, nreset (active low), AXI-Stream interface(s))
4. Validate Block Design
5. Close Block Design

Users may use `hw/bd_src/dfx_region/dfx_region.tcl` as a reference.

4.3 Board-Specific Script

In the current version, only the KV260 board can be fully synthesized and implemented. If the *board* parameter is set to *custom*, the hardware build will stop at the DFX block design with DFX Unified and partial connections; the PS and interconnect will be missing.

To define a new board, users must create or modify the following files:

1. `board_file`: users must create the file at `hw/build_script/<your board name>/board_build.tcl`
   The purpose of this file is to provide a processor block, interconnect block, interrupt system, and other relevant IPs in the Vivado block design. Users can use `hw/build_script/kv260/board_build.tcl` or the GUI in board *custom* as a reference.

2. `constraint_file`: users must create the file at `hw/build_script/<your board name>/constraint.xdc` The purpose of this file is to define the boundaries of the partial reconfigurable region and its resources. Users can use `hw/build_script/kv260/constraint.xdc` as a reference.

3. `build_script`: users must modify the file at `hw/build_script/build.tcl` to invoke the block design creation procedure. Users may use the kv260 option as a reference.

The DFX4ML-ARCH authors would appreciate pull requests for board-specific files.

References

[1] Dfx controller v1.0 product guide (pg374). https://docs.amd.com/v/u/en-US/pg374-dfx-controller, 2026.